IFCT0109 - MF0489_3 - Unidad Didactica 3

COMUNICACIONES SEGURAS

Cuaderno de Practicas Completo

SSH · OpenVPN · stunnel · TLS · Wireshark

Kali Linux vía Guacamole | Docker disponible

Alumno/a: _____

Fecha: _____

Grupo: _____

Calificacion: _____

INFO Este cuaderno contiene 6 practicas sobre comunicaciones seguras. Todas se realizan en tu clon Kali Linux accesible desde juanprofesor.com. Docker ya esta instalado y disponible.

Prácticas de la unidad

N	Practica	Tiempo	Nivel
1	SSH — Autenticación por clave publica y tunneling	45 min	Guiado
2	Análisis de tráfico TLS con tshark y Wireshark	30 min	Guiado
3	Servidor HTTPS con stunnel — añadir TLS a un servicio	30 min	Guiado
4	OpenVPN con Docker — servidor y cliente VPN completo	60 min	Autónomo
5	IPSec con strongSwan entre dos contenedores	45 min	Autónomo
6	Comparativa de tecnologías VPN — análisis practico	30 min	Autónomo

Practica 1 SSH — Autenticación por clave publica y tunneling

Tiempo: 45 min
Guiado

SSH es la herramienta fundamental de administración remota segura. En esta práctica dominamos la autenticación por clave publica y las tres técnicas de tunneling.

1.1 Generar par de claves SSH

```
# Generar par de claves RSA 4096 bits
ssh-keygen -t rsa -b 4096 -C 'alumnoXX@lab' -f ~/.ssh/id_rsa_lab

# Ver las claves generadas
ls -la ~/.ssh/
cat ~/.ssh/id_rsa_lab.pub

# También podemos generar con algoritmo moderno Ed25519
ssh-keygen -t ed25519 -C 'alumnoXX@lab' -f ~/.ssh/id_ed25519_lab
cat ~/.ssh/id_ed25519_lab.pub
```

1.2 Configurar autenticación por clave publica

```
# En Kali podemos simular un servidor SSH con Docker
docker run -d --name ssh-server \
  -p 2222:22 \
  -e SSH_ENABLE_ROOT=true \
  linuxserver/openssh-server 2>/dev/null || \
docker run -d --name ssh-server \
  -p 2222:22 \
  rastasheep/ubuntu-sshd

# Copiar la clave publica al servidor
ssh-copy-id -i ~/.ssh/id_rsa_lab.pub -p 2222 root@localhost

# Conectar sin contraseña
ssh -i ~/.ssh/id_rsa_lab -p 2222 root@localhost
exit
```

NOTA Si el servidor SSH Docker no está disponible, puedes practicar los tunneling contra cualquier servidor SSH accesible en el lab: el instructor tiene un servidor SSH en 192.168.99.1 puerto 22.

1.3 Local Port Forwarding

Permite acceder a un servicio remoto como si fuera local:

```
# Escenario: hay un servicio web en el servidor SSH en el puerto 80
# pero solo accesible desde el propio servidor (no desde fuera)

# Levantar servicio web de prueba en el contenedor
docker exec ssh-server sh -c 'echo "<h1>Servicio interno</h1>" >
/tmp/index.html && cd /tmp && python3 -m http.server 8080 &' 2>/dev/null

# Crear túnel local: puerto 9090 local -> puerto 8080 en el servidor SSH
ssh -i ~/.ssh/id_rsa_lab -p 2222 \
  -L 9090:localhost:8080 \
```

```
  -N -f root@localhost

# Verificar que funciona el túnel
curl http://localhost:9090

# Ver el proceso del túnel
ps aux | grep 'ssh -i'
```

```
<h1>Servicio interno</h1>
# El servicio interno ahora es accesible en localhost:9090
```

1.4 Dynamic Port Forwarding — Proxy SOCKS5

```
# Crear proxy SOCKS5 en el puerto 1080
ssh -i ~/.ssh/id_rsa_lab -p 2222 \
  -D 1080 \
  -N -f root@localhost

# Verificar que escucha
ss -tlnp | grep 1080

# Usar el proxy para acceder a internet a traves del SSH
curl --socks5 localhost:1080 https://ifconfig.me
# La IP mostrada será la del servidor SSH, no la tuya

# Verificar tráfico en tshark
sudo tshark -i lo -f 'port 2222' -c 20 2>/dev/null &
curl --socks5 localhost:1080 http://example.com > /dev/null
# Solo veremos trafico SSH cifrado, no HTTP
```

1.5 Preguntas

1. Explica la diferencia entre autenticacion por password y por clave pública. ¿Por qué es más segura la clave publica?

Tu respuesta:

2. Con el proxy SOCKS5 activo, ¿qué tráfico ve alguien que capture el enlace entre tu maquina y el servidor SSH? ¿Qué tráfico NO puede ver?

Lo que VE: Lo que NO puede ver:

Practica 2 Análisis de tráfico TLS con tshark

Tiempo: 30 min
Guiado

Vamos a capturar y analizar el handshake TLS para entender cómo se negocia una conexión segura en la práctica.

2.1 Capturar el handshake TLS

```
# Instalar tshark si no está disponible
sudo apt install -y tshark 2>/dev/null

# Capturar tráfico TLS en background
sudo tshark -i eth0 \
  -f 'port 443' \
  -w /tmp/captura_tls.pcap \
  -a duration:10 &

# Generar tráfico HTTPS
curl -s https://www.example.com > /dev/null
curl -s https://www.google.com > /dev/null

# Esperar captura
sleep 12

# Analizar la captura
tshark -r /tmp/captura_tls.pcap \
  -Y 'tls.handshake' \
  -T fields \
  -e frame.number \
  -e ip.src \
  -e ip.dst \
  -e tls.handshake.type \
  -e tls.handshake.version
```

2.2 Identificar los mensajes del handshake

```
# Ver solo los mensajes del handshake TLS
tshark -r /tmp/captura_tls.pcap \
  -Y 'tls.handshake.type' \
  -T fields \
  -e frame.time_relative \
  -e tls.handshake.type \
  2>/dev/null | head -20

# Los tipos de handshake:
# 1 = ClientHello
# 2 = ServerHello
# 11 = Certificate
# 12 = ServerKeyExchange
# 14 = ServerHelloDone
# 16 = ClientKeyExchange
# 20 = Finished
```

2.3 Extraer el cipher suite negociado

```
# Ver la versión de TLS y cipher suite usados
```

```
tshark -r /tmp/captura_tls.pcap \
  -Y 'tls.handshake.type == 2' \
  -T fields \
  -e tls.handshake.version \
  -e tls.handshake.ciphersuite \
  2>/dev/null

# Ver con openssl que cipher suite usa un servidor especifico
openssl s_client -connect www.google.com:443 2>/dev/null | \
  grep -E 'Protocol|Cipher'

# Forzar TLS 1.2 y ver diferencia
openssl s_client -connect www.google.com:443 -tls1_2 2>/dev/null | \
  grep -E 'Protocol|Cipher'
```

2.4 Preguntas

1. Anota la secuencia de mensajes del handshake que has capturado y el tiempo entre ClientHello y Finished:

Secuencia de mensajes (con timestamps): Tiempo total del handshake:

2. ¿Qué cipher suite ha negociado tu conexión con google.com? Identifica cada componente (protocolo, cifrado, modo, hash):

Cipher suite completo: Protocolo: Cifrado simetrico: Modo: Hash/MAC:

Practica 3 Servidor HTTPS con stunnel — añadir TLS a un servicio

Tiempo: 30 min
Guiado

stunnel permite añadir cifrado TLS a cualquier servicio que no lo tenga de forma nativa. Vamos a convertir un servidor HTTP simple en HTTPS.

3.1 Crear el servidor HTTP sin cifrar

```
mkdir -p ~/lab-stunnel && cd ~/lab-stunnel

# Crear página web de prueba
mkdir -p web
cat > web/index.html << 'EOF'
<html><body>
<h1>Servidor HTTP sin cifrar</h1>
<p>Este tráfico es visible para cualquiera</p>
</body></html>
EOF

# Levantar servidor HTTP en puerto 8080
docker run -d --name web-sin-tls \
  -p 8080:80 \
  -v $(pwd)/web:/usr/share/nginx/html \
  nginx:alpine

# Verificar que funciona sin cifrar
curl http://localhost:8080

# Capturar tráfico sin cifrar (se ve el contenido)
sudo timeout 5 tshark -i lo -f 'port 8080' -c 5 2>/dev/null &
curl http://localhost:8080 > /dev/null
```

3.2 Generar certificado para stunnel

```
cd ~/lab-stunnel

# Generar clave y certificado autofirmado para stunnel
openssl req -new -x509 -days 365 \
  -nodes \
  -out stunnel.crt \
  -keyout stunnel.key \
  -subj '/C=ES/O=Lab/CN=localhost'

# Combinar en un solo fichero (formato PEM para stunnel)
cat stunnel.key stunnel.crt > stunnel.pem
chmod 600 stunnel.pem
ls -lh stunnel.*
```

3.3 Configurar y lanzar stunnel

```
cd ~/lab-stunnel

# Crear configuracion de stunnel
cat > stunnel.conf << 'EOF'
; Configuracion stunnel - proxy TLS
```

```
foreground = yes
pid = /tmp/stunnel.pid

[https-proxy]
; stunnel escucha en 8443 con TLS
; y reenvía al servidor HTTP en 8080
accept  = 8443
connect = 127.0.0.1:8080
cert    = /etc/stunnel/stunnel.pem
EOF

# Lanzar stunnel con Docker
docker run -d --name stunnel-proxy \
  --network host \
  -v $(pwd)/stunnel.conf:/etc/stunnel/stunnel.conf \
  -v $(pwd)/stunnel.pem:/etc/stunnel/stunnel.pem \
  dweomer/stunnel

# Esperar arranque
sleep 3
docker ps | grep stunnel
```

3.4 Verificar HTTPS con stunnel

```
# Acceder vía HTTPS (ignorar error de certificado autofirmado)
curl -sk https://localhost:8443

# Ver el certificado que presenta stunnel
openssl s_client -connect localhost:8443 2>/dev/null | \
  openssl x509 -noout -subject -issuer -dates

# Comparar tráfico: HTTP sin cifrar vs HTTPS con stunnel
echo '=== Trafico HTTP (visible) ==='
sudo timeout 3 tshark -i lo -f 'port 8080' -c 5 -Y 'http' 2>/dev/null | head -
5

echo '=== Trafico HTTPS (cifrado) ==='
sudo timeout 3 tshark -i lo -f 'port 8443' -c 10 2>/dev/null | head -5
curl -sk https://localhost:8443 > /dev/null
```

3.5 Preguntas

1. ¿En qué casos de uso real seria útil stunnel? Menciona dos ejemplos concretos:

Caso 1: **Caso 2:**

2. ¿Qué diferencia hay entre el tráfico capturado en puerto 8080 y el de 8443? ¿Qué información es visible en cada caso?

Puerto 8080 (HTTP): lo que se ve... Puerto 8443 (HTTPS): lo que se ve...

Practica 4 OpenVPN con Docker — servidor y cliente VPN completo

Tiempo: 60 min
Autonomo

Desplegamos un servidor OpenVPN completo usando Docker y nos conectamos como cliente. Verificaremos que todo el tráfico pasa cifrado por el túnel.

ESCENARIO	Eres el administrador de red de la Clínica Dental Mediterráneo. Los médicos necesitan acceder de forma segura a la intranet desde sus casas. Vas a desplegar un servidor OpenVPN y configurar un cliente.

4.1 Preparar el servidor OpenVPN

```
mkdir -p ~/lab-openvpn && cd ~/lab-openvpn

# Usar imagen kylemanna/openvpn (la más popular)
OVPN_DATA='ovpn-data'

# Inicializar la configuración del servidor
docker run -v ${OVPN_DATA}:/etc/openvpn --rm \
  kylemanna/openvpn ovpn_genconfig \
  -u udp://127.0.0.1

# Inicializar la PKI interna de OpenVPN
docker run -v ${OVPN_DATA}:/etc/openvpn --rm \
  -it kylemanna/openvpn ovpn_initpki
# Cuando pida contrasena: lab2025
# Cuando pida Common Name: OpenVPN-Lab
```

4.2 Generar certificado para el cliente

```
# Crear certificado para el cliente 'medico01'
docker run -v ${OVPN_DATA}:/etc/openvpn --rm \
  -it kylemanna/openvpn easyrsa build-client-full medico01 nopass

# Exportar el perfil completo del cliente (.ovpn)
docker run -v ${OVPN_DATA}:/etc/openvpn --rm \
  kylemanna/openvpn ovpn_getclient medico01 > medico01.ovpn

# Ver el contenido del perfil
cat medico01.ovpn | head -20
wc -l medico01.ovpn
```

4.3 Arrancar el servidor OpenVPN

```
# Arrancar el servidor OpenVPN
docker run -v ${OVPN_DATA}:/etc/openvpn -d \
  --name openvpn-server \
  -p 1194:1194/udp \
  --cap-add=NET_ADMIN \
  kylemanna/openvpn

# Verificar que arranca correctamente
sleep 3
docker logs openvpn-server | tail -10
```

```
# Ver la IP de la red VPN
docker exec openvpn-server ip addr show tun0 2>/dev/null || \
  docker logs openvpn-server | grep 'Initialization Sequence Completed'
```

4.4 Conectar el cliente OpenVPN

```
cd ~/lab-openvpn

# Verificar IP actual ANTES de la VPN
echo '=== IP antes de VPN ==='
ip addr show eth0 | grep 'inet '
ip route | grep default

# Conectar como cliente en background
sudo openvpn --config medico01.ovpn --daemon --log /tmp/openvpn-client.log

# Esperar conexion
sleep 8

# Verificar IP después de la VPN
echo '=== IP después de VPN ==='
ip addr show tun0 2>/dev/null
ip route

# Ver log de conexión
tail -5 /tmp/openvpn-client.log
```

```
=== IP antes de VPN ===
inet 192.168.99.101/24
default via 192.168.99.1

=== IP después de VPN ===
inet 192.172.0.6/24  <- IP dentro del túnel VPN
default via 192.172.0.1  <- ruta por el túnel
```

4.5 Verificar que el trafico va cifrado

```
# Capturar tráfico en la interfaz física
sudo timeout 8 tshark -i eth0 -f 'port 1194' -c 20 2>/dev/null &

# Generar trafico de prueba
curl -s http://example.com > /dev/null
ping -c3 8.8.8.8 > /dev/null

sleep 10

# El tráfico capturado debe ser UDP puerto 1194 (OpenVPN) cifrado
# NO debe verse HTTP ni ICMP en claro

# Desconectar VPN
sudo pkill openvpn
sleep 2
ip route | grep default
# La ruta default debe volver a 192.168.99.1
```

4.6 Preguntas

1. ¿Qué IP obtiene la interfaz tun0 y que significa esa IP en el contexto de la VPN?

IP tun0: Significado:

2. ¿Qué protocolo y puerto usa OpenVPN por defecto? ¿Por qué UDP es preferible a TCP para VPN?

Protocolo y puerto: Por qué UDP es mejor para VPN:

3. ¿Qué archivo es el más importante para un cliente OpenVPN y que contiene?

Archivo: .ovpn Contenido:

Practica 5 IPSec con strongSwan entre dos contenedores

Tiempo: 45 min
Autonomo

Simulamos una VPN IPSec site-to-site entre dos sedes usando strongSwan en contenedores Docker. Cada contenedor representa una sede de la Clínica Dental Mediterráneo.

ESCENARIO	La clínica tiene dos sedes: la sede central en Malaga (red 10.0.1.0/24) y una sucursal en Marbella (red 10.0.2.0/24). Necesitan comunicarse de forma segura.

5.1 Crear la red de laboratorio

```
mkdir -p ~/lab-ipsec && cd ~/lab-ipsec

# Crear red Docker para simular Internet entre las dos sedes
docker network create --subnet=172.20.0.0/24 red-internet-lab 2>/dev/null ||
true
docker network create --subnet=10.0.1.0/24 red-malaga 2>/dev/null || true
docker network create --subnet=10.0.2.0/24 red-marbella 2>/dev/null || true

echo 'Redes creadas:'
docker network ls | grep -E 'internet-lab|malaga|marbella'
```

5.2 Levantar los gateways IPSec

```
# Gateway sede Malaga
docker run -d --name gw-malaga \
  --network red-internet-lab --ip 172.20.0.10 \
  --cap-add NET_ADMIN --cap-add SYS_MODULE \
  --sysctl net.ipv4.ip_forward=1 \
  --sysctl net.ipv4.conf.all.accept_redirects=0 \
  --sysctl net.ipv4.conf.all.send_redirects=0 \
  alpine sh -c 'apk add strongswan iptables && tail -f /dev/null'

# Gateway sede Marbella
docker run -d --name gw-marbella \
  --network red-internet-lab --ip 172.20.0.20 \
  --cap-add NET_ADMIN --cap-add SYS_MODULE \
  --sysctl net.ipv4.ip_forward=1 \
  --sysctl net.ipv4.conf.all.accept_redirects=0 \
  --sysctl net.ipv4.conf.all.send_redirects=0 \
  alpine sh -c 'apk add strongswan iptables && tail -f /dev/null'

# Esperar instalacion
sleep 15
docker ps | grep -E 'malaga|marbella'
```

5.3 Configurar IPSec en gateway Malaga

```
# Crear configuracion IPSec para Malaga
docker exec gw-malaga sh -c 'cat > /etc/ipsec.conf << EOF
config setup
    charondebug="ike 2, knl 2"

conn malaga-marbella
```

```
    # Identidad local
    left=172.20.0.10
    leftsubnet=10.0.1.0/24
    leftid=@malaga
    # Identidad remota
    right=172.20.0.20
    rightsubnet=10.0.2.0/24
    rightid=@marbella
    # Autenticacion PSK
    authby=secret
    # Parametros IKEv2
    keyexchange=ikev2
    ike=aes256-sha256-modp2048
    esp=aes256-sha256
    # Modo tunel automatico
    auto=start
    type=tunnel
EOF'

# Configurar clave precompartida (PSK)
docker exec gw-malaga sh -c 'echo "@malaga @marbella : PSK
\"ClaveSecretaLab2025\"" > /etc/ipsec.secrets'
```

5.4 Configurar IPSec en gateway Marbella

```
# Configuracion simetrica para Marbella
docker exec gw-marbella sh -c 'cat > /etc/ipsec.conf << EOF
config setup
    charondebug="ike 2, knl 2"

conn malaga-marbella
    left=172.20.0.20
    leftsubnet=10.0.2.0/24
    leftid=@marbella
    right=172.20.0.10
    rightsubnet=10.0.1.0/24
    rightid=@malaga
    authby=secret
    keyexchange=ikev2
    ike=aes256-sha256-modp2048
    esp=aes256-sha256
    auto=start
    type=tunnel
EOF'

docker exec gw-marbella sh -c 'echo "@marbella @malaga : PSK
\"ClaveSecretaLab2025\"" > /etc/ipsec.secrets'
```

5.5 Establecer el tunel IPSec

```
# Arrancar strongSwan en ambos gateways
docker exec gw-malaga ipsec start
docker exec gw-marbella ipsec start

# Esperar establecimiento del tunel
sleep 5

# Verificar estado del tunel en Malaga
```

```
docker exec gw-malaga ipsec status

# Debe mostrar: malaga-marbella{1}: INSTALLED, TUNNEL

# Ver las SA (Security Associations) establecidas
docker exec gw-malaga ipsec statusall | grep -E 'IKE|ESP|INSTALLED'
```

5.6 Preguntas

1. ¿Qué es una SA (Security Association) en IPSec y que parámetros negocia IKEv2?

Tu respuesta:

2. ¿Por qué se usan dos SAs (una para cada dirección) en una conexión IPSec?

Tu respuesta:

Practica 6 Comparativa de tecnologías VPN — análisis practico

Tiempo: 30 min
Autonomo

En esta última practica analizas y documentas las diferencias practicas entre las tecnologías que has usado, con datos reales de tus propias pruebas.

6.1 Medir rendimiento de cada tecnologia

```
# Instalar herramienta de medición
sudo apt install -y iperf3 2>/dev/null || true

# SIN VPN - tráfico directo
echo '=== Prueba sin VPN ==='
time curl -s http://ipv4.download.thinkbroadband.com/1MB.zip -o /dev/null

# CON SSH Tunnel (si tienes servidor SSH activo)
echo '=== Prueba con SSH tunnel ==='
ssh -L 8888:ipv4.download.thinkbroadband.com:80 -N -f root@localhost -p 2222
2>/dev/null
time curl -s http://localhost:8888/1MB.zip -o /dev/null 2>/dev/null || echo
'SSH tunnel: medir latencia con ping'

# Medir latencia con y sin VPN
echo '=== Latencia sin VPN ==='
ping -c 10 8.8.8.8 | tail -2

# Iniciar OpenVPN y medir latencia
sudo openvpn --config ~/lab-openvpn/medico01.ovpn --daemon --log
/tmp/ovpn2.log 2>/dev/null
sleep 8
echo '=== Latencia con OpenVPN ==='
ping -c 10 8.8.8.8 | tail -2
sudo pkill openvpn 2>/dev/null
```

6.2 Tabla comparativa — rellena con tus resultados

Criterio	SSH Tunnel	OpenVPN	IPSec	stunnel+TLS
Tiempo configuracion				
Facilidad de uso				
Latencia medida				
Todo el trafico				
Atraviesa firewalls				
Caso de uso ideal				

6.3 Informe de recomendación

La Clínica Dental Mediterráneo necesita elegir una tecnologia VPN para sus tres casos de uso. Basándote en tus prácticas, haz una recomendación justificada:

CASO 1 - Teletrabajo de médicos desde casa: Tecnología recomendada: Justificación:

CASO 2 - Conexión permanente sede Malaga-Marbella: Tecnología recomendada: Justificación:

CASO 3 - Acceso puntual del administrador a un servidor interno: Tecnología recomendada: Justificación: